Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- B-ağacının (B-tree) özellikleri (ekrandan, 00:10:00 ve 00:13:50)
  - B-ağaçları disk işlemleri için, diskteki verilere hızlı erişmek ve arama yapmak için kullanılır. Kırmızı-siyah ağaçlar gibi dengelidir, ama bir düğümün ikiden fazla çocuğu olabilir.
  - Her düğümdeki anahtar sayısı disk sayfasının (page) boyutuyla orantılıdır; böylece bir düğüm tek disk erişimiyle okunur. Kökün ana bellekte (main memory) tutulduğu varsayılır.
  - (1) Her x düğümünde `x.n` (düğümdeki anahtar sayısı) tutulur. (2) Anahtarlar düğüm içinde sıralıdır: $x.key_1 \leq x.key_2 \leq ... \leq x.key_{x.n}$; ayrıca `x.leaf` adlı mantıksal (boolean) alan düğümün yaprak olup olmadığını tutar. Her iç düğümün `x.n + 1` çocuğu vardır: $x.c_1, ..., x.c_{x.n+1}$.
  - (3) Anahtarlar alt ağaçlardaki anahtar aralıklarını ayırır: k_i, $x.c_i$ alt ağacındaki herhangi bir anahtarsa $k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq k_3 \leq ...$
  - (4) Bütün yapraklar aynı derinliktedir; bu derinlik ağacın yüksekliği h'dir.
  - (5) $t \geq 2$ olan bir sabit seçilir: B-ağacının en küçük derecesi (minimum degree). Kök dışındaki her düğümde en az $t-1$ anahtar, her iç düğümde en az t çocuk olmalıdır. Kökte en az 1 anahtar olur. Her düğümde (kök dahil) en fazla $2t-1$ anahtar, yani iç düğümde en fazla $2t$ çocuk olur. $2t-1$ anahtarlı düğüme dolu (full) denir.
- B-ağacının yüksekliği teoremi (ekrandan, 00:20:00 ve 00:26:45)
  - $n \geq 1$ ise, n anahtarlı, en küçük derecesi $t \geq 2$ olan her B-ağacının yüksekliği için $h \leq \log_t \frac{n+1}{2}$.
  - En uzun ağaç her düğümün mümkün olan en az anahtarı tuttuğu ağaçtır: kökte 1 anahtar ve 2 çocuk, diğer düğümlerde $t-1$ anahtar ve t çocuk. Derinlik 1'de 2, derinlik 2'de $2t$, derinlik 3'te $2t^2$ düğüm; genelde derinlik i'de $2t^{i-1}$ düğüm.
  - $n \geq 1 + (t-1)\sum_{i=1}^{h} 2t^{i-1} = 2t^h - 1$, buradan $n+1 \geq 2t^h$ ve $\log_t \frac{n+1}{2} \geq h$.
  - İkili ağaçlarda ikiye bölünme olduğu için 2 tabanlı logaritma çıkıyordu; burada t'ye bölünme olduğu için t tabanlı logaritma çıkar.
- `B-TREE-SEARCH(x, k)` (ekrandan, 00:31:15 ve 00:33:45)
  - İki aşamalıdır: düğüm içinde `while` döngüsüyle soldan sağa tarama, sonra uygun çocuğa özyinelemeli (recursive) iniş.
  - `while` döngüsü $k \leq x.key_i$ olan en küçük i'yi bulur. $i \leq x.n$ ve $k = x.key_i$ ise `(x, i)` döner. `x.leaf` ise `NIL` döner. Değilse `DISK-READ(x.c_i)` yapılır ve `B-TREE-SEARCH(x.c_i, k)` çağrılır.
  - Karmaşıklık: düğüm içi tarama en fazla $2t-1$ anahtar, yani $O(t)$; iniş ağacın derinliği kadar, yani $O(\log_t n)$. Toplam $O(t \log_t n)$.
- `B-TREE-CREATE(T)` (ekrandan, 00:40:00)
  - `ALLOCATE-NODE()` ile boş düğüm alınır (disk okuması gerekmez). `x.leaf = TRUE`, `x.n = 0`, `DISK-WRITE(x)`, `T.root = x`.
- `B-TREE-SPLIT-CHILD(x, i)` (ekrandan, 00:50:00)
  - x'in i'inci çocuğunu böler; bölünen düğüm değil, onun ebeveyni parametre olarak verilir.
  - Dolu $y = x.c_i$ ($2t-1$ anahtar) iki düğüme ayrılır: her birinde $t-1$ anahtar kalır, ortanca (median) anahtar $y.key_t$ ebeveyn x'e çıkar.
  - Yeni z düğümüne y'nin son $t-1$ anahtarı ve (yaprak değilse) son t çocuğu kopyalanır;

x'in çocuk ve anahtar dizileri birer kaydırılır, $x.c_{i+1} = z$, $x.key_i = y.key_t$, `x.n` bir artırılır; y, z, x diske yazılır.
  - Örnek ($t = 4$): y = P Q R S T U V. Bölünce y = P Q R, z = T U V; S, x'te N ile W arasına çıkar.
- `B-TREE-INSERT(T, k)` ve `B-TREE-INSERT-NONFULL(x, k)` (ekrandan, 01:13:32 ve 01:22:35)
  - Kök doluysa ($r.n = 2t - 1$) yeni düğüm s kök yapılır, `s.leaf = FALSE`, `s.n = 0`, `s.c_1 = r`, `B-TREE-SPLIT-CHILD(s, 1)` ve `B-TREE-INSERT-NONFULL(s, k)` çağrılır. Değilse `B-TREE-INSERT-NONFULL(r, k)` çağrılır.
  - Örnek ($t = 4$): kök r = A D F H L N P doluysa H yeni köke çıkar; A D F ve L N P iki çocuk olur.
  - `INSERT-NONFULL`, verilen düğümün dolu olmadığını varsayar. Yapraksa büyük anahtarlar birer sağa kaydırılır ve k boşluğa yazılır. Yaprak değilse doğru çocuk bulunur, okunur; o çocuk doluysa önce bölünür, sonra k'nin hangi yarıya gideceği seçilir ve özyineleme sürer.
  - B-ağacında ekleme her zaman yaprağa yapılır; özyinelemenin çıkış noktası yapraktır.
- Ekleme örneği, $t = 3$ (ekrandan, 01:24:20 ve 01:30:00)
  - Başlangıç: kök G M P X; yapraklar A C D E, J K, N O, R S T U V, Y Z.
  - B: kök ve ilgili yaprak dolu değil → A B C D E.
  - Q: ilgili yaprak R S T U V dolu → bölünür, T köke çıkar (G M P T X); Q, Q R S yaprağına eklenir.
  - L: kök G M P T X dolu → kök bölünür, P yeni kök olur (G M ve T X); L, J K L yaprağına eklenir.
  - F: yaprak A B C D E dolu → bölünür, C yukarı çıkar (C G M); F, D E F yaprağına eklenir.
- B-ağacından silme (ekrandan, 01:35:05 ve 01:37:28)
  - Eklemede $2t - 1$'in aşılmaması, silmede $t - 1$'in altına düşülmemesi gözetilir. Yalnız ilgili düğüme değil, kardeşlerine, ebeveynine ve ebeveynin kardeşlerine de bakılır. Silme de özyinelemeli yapılır; kitapta alt durumlarıyla 8 kural vardır.
  - Kitap örneği ($t = 3$), başlangıç: kök P; C G M ve T X; yapraklar A B, D E F, J K L, N O, Q R S, U V, Y Z.
  - F (durum 1): yapraktan doğrudan silinir.
  - M (durum 2a): öncülü (predecessor) L, M'nin yerine geçer.
  - G (durum 2c): G'nin iki çocuğu (D E, J K) en az anahtar sayısında olduğu için birleştirilir: D E J K.
  - D (durum 3b): C L ve T X en az sayıda olduğu için P aşağı indirilip birleştirilir: C L P T X; ağacın yüksekliği bir azalır.
  - B (durum 3a): kardeş E J K'de fazladan anahtar olduğu için E köke çıkar, C aşağı iner; sonuç A C, kök E L P T X.
- Yığın (heap) (ekrandan, 01:44:20)
  - Neredeyse tam (nearly complete) ikili ağaçtır; ikili arama ağacı değildir. Öncelik kuyruklarının (priority queue) gerçeklenmesinde kullanılır.
  - Genelde dizide tutulur: $A[1 \mathinner{..} A.length]$, $0 \leq A.heap_size \leq A.length$; geçerli elemanlar $A[1 \mathinner{..} A.heap_size]$, kök $A[1]$.
  - Ebeveyn: indisi ikiye bölmek; sol çocuk: ikiyle çarpmak; sağ çocuk: ikiyle çarpıp bir eklemek. Bunlar bit kaydırma ve makrolarla hızlı yazılabilir.
  - Max-heap özelliği: $A[PARENT(i)] \geq A[i]$, en büyük eleman köktedir. Min-heap: $A[PARENT(i)] \leq A[i]$, en küçük eleman köktedir.
- `MAX-HEAPIFY(A, i)` (ekrandan, 01:52:08 ve 01:55:30)
  - i'nin sol ve sağ alt ağaçlarının max-heap olduğu varsayılır. $A[i]$, $A[l]$, $A[r]$ arasından en büyük (`largest`) bulunur; `largest` $\neq i$ ise $A[i]$ ile $A[largest]$ değiştirilir ve `MAX-HEAPIFY(A, largest)` çağrılır.
  - Örnek `MAX-HEAPIFY(A, 2)`: 4 ile 14 değişir, sonra 4 ile 8 değişir; sonuç 16, 14, 10, 8, 7, 9, 3, 2, 4, 1.
- `BUILD-MAX-HEAP(A)` (ekrandan, 02:00:00 ve 02:10:00)
  - `A.heap_size = A.length`; `for i = ⌊A.length/2⌋ downto 1`: `MAX-HEAPIFY(A, i)`. $\lfloor n/2 \rfloor + 1, \ldots, n$ indisli elemanlar yapraktır, yaprak olmayanlar işlenir.
  - Örnek: 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 → 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 (python ile doğrulandı).

Hocanın Özellikle Vurguladığı Kısımlar

- B-ağacının özellikleri bütün işlemlerde korunmalıdır
  - Ekleme ve silme kodları, düğümlerdeki anahtar sayısını $t-1$ ile $2t-1$ arasında tutmak için tasarlanmıştır.
- Kök istisnadır
  - Kök tek anahtarla kalabilir, diğer düğümler kalamaz; yükseklik kanıtında kökün bu tek anahtarı ayrıca eklenir.
- `B-TREE-SPLIT-CHILD` ebeveyni ve çocuk indisini alır
  - "x'i i'inci anahtardan bölmek" değil, "x'in i'inci çocuğunu bölmek" anlamına gelir.
- Ekleme yaprağa yapılır
  - `INSERT-NONFULL` aşağı inerken karşılaştığı dolu düğümleri böler, böylece yaprağa geldiğinde yer olduğu garanti edilir; yaprakta ayrıca doluluk kontrolü yapılmaz.
- Disk erişimi
  - Arama ve eklemede bir sonraki çocuğa inerken `DISK-READ` gerekir, çünkü bellekte yalnız o anki düğüm tutulabilir. Değişen düğümler `DISK-WRITE` ile yazılır.
- Silmede yüksekliğin azalması
  - D silinirken (durum 3b) kök boşalır ve ağacın yüksekliği bir azalır; hoca bunu yüksekliğin azaldığı tek durum olarak andı.

Kısa Tekrar Notları

- B-ağacı: dengeli, çok çocuklu arama ağacı; düğüm boyu disk sayfasına göre seçilir.
- `x.n` anahtar → `x.n + 1` çocuk; `x.leaf` yaprak bilgisi.
- Kök dışı düğüm: en az $t-1$, en fazla $2t-1$ anahtar; iç düğüm: en az t, en fazla $2t$ çocuk; $t \geq 2$.
- Bütün yapraklar aynı derinlikte.
- $h \leq \log_t \frac{n+1}{2}$; kanıt: $n \geq 2t^h - 1$.
- Arama: $O(t \log_t n)$.
- Bölme: ortanca anahtar ebeveyne çıkar, iki yarıda $t-1$'er anahtar kalır.
- Kök doluysa yeni kök açılır ve kök bölünür.
- Ekleme tek inişte; dolu düğümler inerken bölünür; anahtar yaprağa eklenir.
- Silme: anahtar sayısı $t-1$'in altına düşmemeli; öncülle değiştirme, birleştirme, kardeşten ödünç alma.
- Heap dizide: ebeveyn $\lfloor i/2 \rfloor$, sol $2i$, sağ $2i+1$.
- `MAX-HEAPIFY`: üçün en büyüğünü yukarı al, aşağıda özyinele.
- `BUILD-MAX-HEAP`: $\lfloor n/2 \rfloor$'den 1'e kadar `MAX-HEAPIFY`.

Detaylı Açıklamalar

- B-ağaçları neden var?
  - Veri diskte tutulduğunda her disk erişimi pahalıdır. B-ağacı her düğüme bir disk sayfası kadar anahtar koyarak ağacı kısa tutar; bir düğüm tek erişimle okunur, aramada inilen her seviye bir disk okumasıdır. Kök sürekli bellekte tutulur.
- Hangi t için geçerli? (ekrandan, 00:13:50)
  - Önceki dersteki örnek ağaçta kök M (1 anahtar), iç düğümler D H (2 anahtar) ve 3 anahtarlı bir düğüm; yapraklarda 2 ya da 3 anahtar vardır. $t=2$ için 1-3 anahtar, $t=3$ için 2-5 anahtar gerekir; ikisi de sağlanır. $t=1$ olamaz ($t \geq 2$). $t=4$ için en az 3 anahtar gerekir; 2 anahtarlı düğümler bunu bozar. Ağaç yalnız $t=2$ ve $t=3$ için geçerlidir.
  - Ders sonrası not: Hoca tahtada sağdaki iç düğümü "Q T S" diye çizdi; çocukları V W ve Y Z olduğuna göre üçüncü anahtar önceki dersteki gibi X olmalıdır. Anahtar sayıları değişmediği için cevap aynıdır.
- Yükseklik kanıtının mantığı
  - Yüksekliği en büyük yapmak için her düğüme en az bilgi konur. Kök 1 anahtar ve 2 çocukla başlar; sonraki her düğüm $t-1$ anahtar ve t çocuk taşır. Derinlik $i \geq 1$'de $2t^{i-1}$ düğüm, her birinde $t-1$ anahtar vardır. Toplam anahtar en az $1 + (t -$

$1) \sum_{i=1}^{h} 2t^{i-1}$'dir; geometrik seri toplamıyla bu $2t^h - 1$ olur (python ile doğrulandı). Logaritma alınınca $h \leq \log_t \frac{n+1}{2}$ çıkar.

- Bölme işleminin sözde kodu (ekrandan, 00:50:00)

```
B-TREE-SPLIT-CHILD(x, i)
z = ALLOCATE-NODE()
y = x.c_i
z.leaf = y.leaf
z.n = t - 1
for j = 1 to t - 1
    z.key_j = y.key_{j+t}
if not y.leaf
    for j = 1 to t
        z.c_j = y.c_{j+t}
y.n = t - 1
for j = x.n + 1 downto i + 1
    x.c_{j+1} = x.c_j
x.c_{i+1} = z
for j = x.n downto i
    x.key_{j+1} = x.key_j
x.key_i = y.key_t
x.n = x.n + 1
DISK-WRITE(y)
DISK-WRITE(z)
DISK-WRITE(x)
```

- Eklemenin sözde kodu (ekrandan, 01:13:32 ve 01:22:35)

```
B-TREE-INSERT(T, k)
r = T.root
if r.n == 2t - 1
    s = ALLOCATE-NODE()
    T.root = s
    s.leaf = FALSE
    s.n = 0
    s.c_1 = r
    B-TREE-SPLIT-CHILD(s, 1)
    B-TREE-INSERT-NONFULL(s, k)
else B-TREE-INSERT-NONFULL(r, k)

B-TREE-INSERT-NONFULL(x, k)
i = x.n
if x.leaf
    while i >= 1 and k < x.key_i
        x.key_{i+1} = x.key_i
        i = i - 1
    x.key_{i+1} = k
    x.n = x.n + 1
    DISK-WRITE(x)
else while i >= 1 and k < x.key_i
        i = i - 1
    i = i + 1
    DISK-READ(x.c_i)
    if x.c_i.n == 2t - 1
        B-TREE-SPLIT-CHILD(x, i)
        if k > x.key_i
            i = i + 1
    B-TREE-INSERT-NONFULL(x.c_i, k)
```

  - $t = 3$ örneğindeki dört ekleme python ile adım adım çalıştırıldı; sonuç ağaç kök P, iç düğümler C G M ve T X, yapraklar A B, D E F, J K L, N O, Q R S, U V, Y Z'dir ve

tahtadaki son çizimle aynıdır.

- Silme
  - Hoca silmeyi kod yazmadan kitap şekli üzerinden anlattı. Önemli olan, hiçbir düğümün $t-1$ anahtarın altına düşmemesidir. Kurallar uyulduğu sürece farklı tasarımlar kabul edilebilir.
- Yığının dizide tutulması
  - Ağaç neredeyse tam olduğu için dizi seyrek (sparse) olmaz; işaretçi aritmetiği ve ek hesap yükünden kaçınılır. Örnek max-heap: 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 (ekrandan, 01:44:20). Her alt ağaç kendi içinde düzenlidir, ama dizi bütünüyle sıralı değildir.
- `BUILD-MAX-HEAP` örneğinin adımları (ekrandan, 02:00:00)
  - $i = 5$: 16'nın tek çocuğu 7, değişiklik yok. $i = 4$: 2 ile 14 değişir. $i = 3$: 3 ile 10 değişir. $i = 2$: 1 ile 16 değişir, sonra 1 ile 7 değişir. $i = 1$: 4 ile 16, sonra 4 ile 14, sonra 4 ile 8 değişir.
- Sonraki konu bağlantısı
  - Hoca Fibonacci yığınlarından önce yığını hatırlatmak istedi; Fibonacci yığınlarından önce en küçük kapsayan ağaçtan (minimum spanning tree) da söz edeceğini belirtti.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.